

Elimination of Left Recursion

Compiler Design • Top-down Parsing • Aho–Ullman / Dragon Book lineage

10-slide teaching deck

with worked examples

recursive descent ready

Core idea

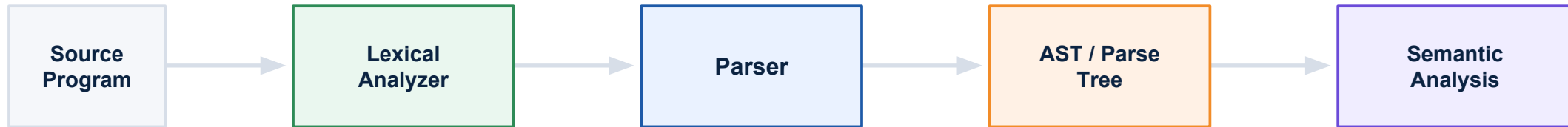
$$A \rightarrow A \alpha \mid \beta$$

⇓

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \mid \varepsilon \end{aligned}$$

1. Where left recursion fits in a compiler

The issue appears in the parser, after lexical analysis and before semantic analysis.



Parser needs a grammar that does not trap top-down parsing in infinite self-calls.

Predictive / recursive-descent parsers expand the leftmost nonterminal first.

A production like $A \rightarrow A\alpha$ asks the parser to call A again before consuming input.

Left recursion removal rewrites the grammar without changing the strings generated.

Important: left recursion is a parser-design problem, not a language-design problem.

2. Why left recursion breaks top-down parsing

Immediate left recursion makes the parser repeat the same call without advancing input.

$A \rightarrow A \alpha \mid \beta$

Recursive-descent skeleton

```
parseA() {  
  parseA(); // A → A α  
  parseAlpha();  
}  
// no token consumed before recursion
```

Input pointer stays at the same token

Result

Infinite recursion / stack overflow

The grammar can be perfectly valid for bottom-up parsing.

The problem arises when a top-down parser tries to expand A before seeing β .

Solution: make β appear first and move repetition to a new helper nonterminal.

3. Immediate left recursion: the standard transformation

This is the core rule used before building LL(1) / predictive parsers.

Before

$$A \rightarrow A \alpha_1 \mid A \alpha_2 \mid \dots \mid \beta_1 \mid \beta_2 \mid \dots$$

After

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \\ A' &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \epsilon \end{aligned}$$

α part

What was after the recursive A. It becomes a repeatable suffix in A'.

β part

A non-recursive start. Every valid derivation must begin here.

ϵ

Allows the repetition to stop after zero or more α suffixes.

Mnemonic: A generates β followed by zero or more α pieces.

4. Worked example: expression grammar

Classic arithmetic grammar, written in the style used in compiler texts.

Original grammar

```
E → E + T | T
T → T * F | F
F → ( E ) | id
```

Transformed grammar

```
E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → ( E ) | id
```

Step logic

E has immediate left recursion: $E \rightarrow E + T \mid T$, so $\alpha = +T$ and $\beta = T$.

T has immediate left recursion: $T \rightarrow T * F \mid F$, so $\alpha = *F$ and $\beta = F$.

F has no left recursion, so it remains unchanged.

5. Derivation after elimination: id + id * id

The transformed grammar supports top-down parsing while preserving precedence.

$E \Rightarrow T E'$
 $\Rightarrow F T' E'$
 $\Rightarrow id T' E'$
 $\Rightarrow id E'$
 $\Rightarrow id + T E'$
 $\Rightarrow id + F T' E'$
 $\Rightarrow id + id T' E'$
 $\Rightarrow id + id * F T' E'$
 $\Rightarrow id + id * id$

How precedence is preserved



E handles +
T handles *
F handles id / parentheses

Why it works

The helper nonterminals E' and T' represent “continue or stop” decisions. This is exactly what a predictive parser can choose using lookahead.

The parse tree shape changes, but the arithmetic language and precedence intent remain.

6. Indirect left recursion

Sometimes A does not call itself immediately, but returns to itself through another nonterminal.

$$\begin{aligned} S &\rightarrow A a \mid b \\ A &\rightarrow S c \mid d \end{aligned}$$

Cycle


$$S \Rightarrow A a \Rightarrow S c a$$

Remove by substitution + immediate elimination

$$\begin{aligned} A &\rightarrow S c \mid d \\ &\rightarrow (A a \mid b) c \mid d \\ &\rightarrow A a c \mid b c \mid d \end{aligned}$$

$$\begin{aligned} A &\rightarrow b c A' \mid d A' \\ A' &\rightarrow a c A' \mid \epsilon \\ S &\rightarrow A a \mid b \end{aligned}$$

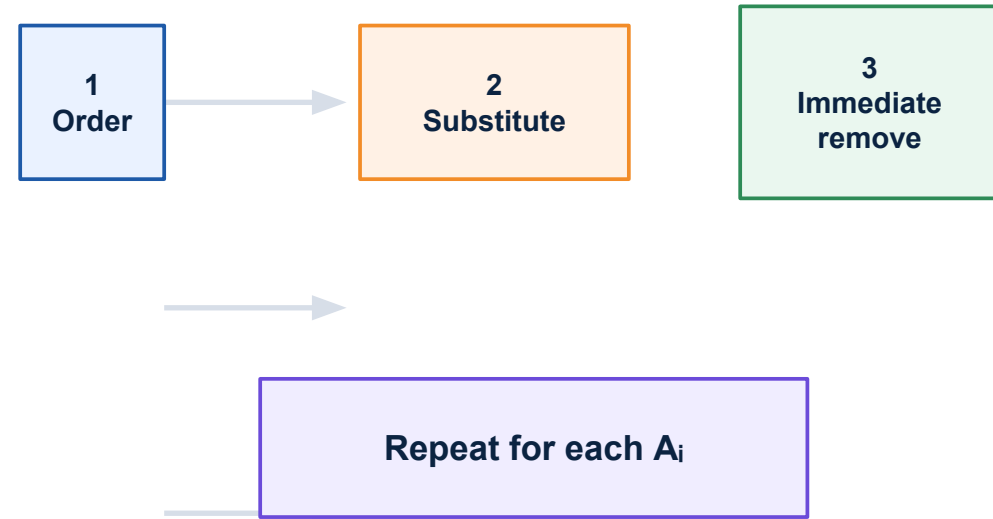
7. General algorithm for eliminating left recursion

Process nonterminals in a fixed order and remove earlier dependencies first.

Aho-style algorithm

Order nonterminals: $A_1, A_2, \dots, A_n$

```
for i = 1 to n:  
  for j = 1 to i-1:  
    replace each  $A_i \rightarrow A_j \gamma$   
    by  $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots$   
    where  $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots$   
  eliminate immediate left recursion on  $A_i$ 
```



Why order matters

By the time A_i is processed, all indirect left recursion through earlier A_j has already been exposed as immediate left recursion.

8. What changes — and what does not

The transformation is language-preserving, but parser behavior and tree shape become friendlier for top-down parsing.

Same language

Strings generated before and after transformation are the same.

Different tree shape

Left-branching trees become right-recursive helper chains.

Parser becomes safe

Recursive descent can consume a β prefix before repeating α .

Common caveats

Associativity may need explicit semantic handling. A right-recursive grammar can produce a different tree shape for left-associative operators.

Eliminating left recursion does not automatically make a grammar LL(1). Left factoring may still be required.

The helper nonterminal A' must include ϵ ; otherwise the grammar cannot stop repeating α .

Rule of thumb: remove left recursion first, then left-factor, then build FIRST/FOLLOW and parsing table.

9. Summary cheat sheet + classroom practice

Use this as the one-slide revision map.

Cheat sheet

$$A \rightarrow A \alpha \mid \beta$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \varepsilon$$

Practice set

1) $A \rightarrow A x \mid y$

2) $E \rightarrow E + T \mid E - T \mid T$

3) $S \rightarrow S a \mid S b \mid c \mid d$

4) $S \rightarrow A a \mid b$
 $A \rightarrow S c \mid d$

Learning outcome

Students should be able to identify immediate/indirect left recursion, apply the Aho-style elimination algorithm, and prepare grammars for predictive parsing.